Assignment Instructions

Listen

CIS 228: Programming Assignment - Digital Sorting Station

🎓 Learning Objectives

Upon successful completion of this assignment, you will be able to:

1. **Implement Core DOM Events:** Correctly attach and manage dragstart, dragover, dragleave, and drop event listeners.
2. **Manage State:** Use dataTransfer to pass custom data (item IDs) between events.
3. **DOM Manipulation:** Clone and append elements dynamically to the Document Object Model (DOM) without altering the source structure.
4. **Modular Code Design:** Write clean, separated JavaScript logic that handles user interaction.

📝 Assignment Description: The Digital Sorting Station

You are tasked with building a simple "Digital Sorting Station." This application simulates a process where an inventory manager must categorize unsorted digital assets (represented by cards) into the correct departmental folders.

Your application must feature:

1. **Source Inventory:** A container holding a list of unsorted items.
2. **Categorization Zones:** Three distinct drop zones, each representing a different department (e.g., "Needs Review," "Complete," "Archived").
3. **Functionality:** Users must be able to drag a card from the Source Inventory and drop it into one of the three zones.

Goal: Write the JavaScript logic that makes the drag-and-drop process functional and visually engaging.

🎯 Detailed Requirements

Your final application must meet the following technical requirements:

1. **Drag Enablement:** All items in the Source Inventory must be marked as draggable.
    - **Visual Feedback:**When an item is dragged, it must visually change (e.g., fade or slightly darken) to indicate it is being held.
    - When the mouse hovers over **any** of the three Categorization Zones, the zone must change its background color and/or border to provide immediate visual feedback that it is a valid drop target.

- **Drop Handling:**The dragover event handler **must** call event.preventDefault() to make the drop zones accept items.
- When an item is dropped, the system must **clone** the item and place the clone into the target zone. **Crucially, the original item card in the Source Inventory must remain untouched.**

1. **Data Persistence:** The ID of the item must be passed and retrieved correctly using the dataTransfer object.

📂 Starter Files (Do Not Change These Structures)

Use the following three files as your starting point. Your primary focus will be writing the logic within script.js and adjusting the styles in style.css to meet the requirements.

1. index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Digital Sorting Station</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>📂 Digital Sorting Station</h1>
    <p>Drag the unsorted assets from the Inventory into the appropriate Category Zone.
</p>

    <div class="container">
        <!-- Source Inventory -->
        <div class="inventory-section">
            <h2>📥 Unsorted Inventory</h2>
            <div class="card draggable-item" draggable="true" data-id="asset_A">
                Project Report Q3
            </div>
            <div class="card draggable-item" draggable="true" data-id="asset_B">
                Client Meeting Notes
            </div>
            <div class="card draggable-item" draggable="true" data-id="asset_C">
                Budget Proposal Draft
            </div>
            <div class="card draggable-item" draggable="true" data-id="asset_D">
                Marketing Strategy V2
            </div>
        </div>

        <!-- Target Zones -->
        <div id="review-zone" class="drop-zone" data-category="Review">
            <h2>🔎 Needs Review</h2>
            <p>Items requiring manager approval.</p>
        </div>
        <div id="complete-zone" class="drop-zone" data-category="Complete">
            <h2>✅ Complete</h2>
            <p>Finalized and ready for archiving.</p>
        </div>
        <div id="archive-zone" class="drop-zone" data-category="Archive">
            <h2><0xF0><0x9F><0x97><0x84> Archived</h2>
            <p>Old or completed records.</p>
        </div>
    </div>
```

```
    <script src="script.js"></script>
</body>
</html>
```

2. style.css

```
body {
    font-family: sans-serif;
    padding: 20px;
    background-color: #f9f9f9;
}

.container {
    display: flex;
    justify-content: space-around;
    gap: 20px;
    max-width: 1400px;
}

.inventory-section, .drop-zone {
    flex: 1;
    padding: 15px;
    border-radius: 10px;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
    min-height: 350px;
}

/* Source Inventory Styling */
.inventory-section {
    background-color: #e0f7fa; /* Light Aqua */
    border: 3px dashed #b2ebf2;
}

/* Drop Zone Styling */
.drop-zone {
    border: 2px solid #ccc;
    transition: background-color 0.3s, box-shadow 0.3s;
}

/* VISUAL REQUIREMENT: Hover Feedback */
.drop-zone.drag-over {
    background-color: #c8e6c9; /* Lighter Green */
    border-color: #4caf50;
    box-shadow: 0 0 15px rgba(76, 175, 80, 0.5);
}

/* Card Styling */
.card {
    background-color: #ffffff;
    padding: 12px;
    margin-bottom: 10px;
```

```
    border-radius: 6px;
    box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);
    cursor: grab;
    transition: opacity 0.2s, transform 0.2s;
}

/* VISUAL REQUIREMENT: Dragging Feedback */
.dragging {
    opacity: 0.3;
    transform: scale(0.99);
    border: 1px dashed #aaa;
}
```

3. script.js

```
// === START CODING HERE ===

// 1. Get references to all required elements
const draggableItems = document.querySelectorAll('.draggable-item');
const dropZones = document.querySelectorAll('.drop-zone');

// 2. Implement the dragstart and dragend handlers for the source items.
//    (Attach these handlers to all items in the 'draggableItems' NodeList)
//    - Must set the item's data-id into the dataTransfer object.
//    - Must add/remove the 'dragging' class for visual feedback.

// 3. Implement the dragover handler for the drop zones.
//    (Attach this handler to all elements in the 'dropZones' NodeList)
//    - CRITICAL: Must call event.preventDefault() and apply the 'drag-over' class.

// 4. Implement the dragleave handler for the drop zones.
//    (Attach this handler to all elements in the 'dropZones' NodeList)
//    - Must remove the 'drag-over' class.

// 5. Implement the drop handler for the drop zones.
//    (Attach this handler to all elements in the 'dropZones' NodeList)
//    - Must retrieve the item ID from the event dataTransfer.
//    - Find the original element using the ID.
//    - Clone the original element (DO NOT MOVE IT).
//    - Append the clone to the current drop zone.

// === END CODING HERE ===
```

Add your completed code here.

Submission

Click to add text

Word count: 0

Drag files here

Maximum size: 512 MB

or

Upload a file **Upload from cloud**

Save and Close **Submit**